

LEGALEASE — CODE LAYOUT, READABILITY, REUSABILITY

FOLDER STRUCTURE:

LegalEase/

- ├── ai_core/ ← AI logic (reusable class)
- ├── legalEaseAPI/ ← FastAPI backend (modular routes)
- ├── frontend/ ← Streamlit UI (separate concern)
- └── config.py ← Central configuration

SEPARATION OF CONCERNS:

- gemini_generator.py: Only talks to Gemini (reusable in any project)
- routes.py: Only defines API endpoints (thin layer)
- main.py: Only initializes FastAPI (minimal logic)
- app.py (frontend): Only handles UI and HTTP calls

READABILITY PRACTICES:

- Descriptive function names (generate_document, not gen_doc)
- Docstrings on every function
- Type hints where helpful
- Constants at top (MODEL_NAME)
- No magic numbers or strings

REUSABILITY:

- GeminiDocumentGenerator class can be imported into any project
- Format functions (format_docx, format_pdf) are standalone
- Config.py centralizes API key and model name
- requirements.txt pins all dependencies

NAMING CONVENTIONS:

- Files: snake_case (gemini_generator.py)
- Classes: PascalCase (GeminiDocumentGenerator)
- Functions: snake_case (generate_document)
- Constants: UPPER_CASE (MODEL_NAME, GEMINI_API_KEY)